

Practice 2.

Objectives:

1. Learn to clean data with pandas.
2. Do a simple Linear regression

Aplication:

Data base: Data set from the UCI Machine Learning Repository (<http://archive.ics.uci.edu/ml>)

1. Cars data: <http://archive.ics.uci.edu/ml/datasets/Automobile>.
 - Predict **price** in dollars.
 2. Database: Wine quality, <http://archive.ics.uci.edu/ml/datasets/Wine+Quality>
 - Predict **quality** (score between 0 and 10)
-

First we load the Libraries

- Pandas: For dealing with tabulated data: excel + sql
- sklearn: Machine learning framework
- os: anciliary functions for dealing with the operating system

```
In [195... import pandas as pd
import os
```

```
In [196... import matplotlib.pyplot as plt
import numpy as np
import sklearn
```

Use the os library for generating the path to the file

```
In [197... File = "Bdcars.csv"
Filename = os.path.join(os.getcwd(), 'Data', File)
print(f'Filename with path: \n {Filename}')
```

Filename with path:
c:\mlearn-lab\SecondPractice\Data\Bdcars.csv

First rows of the file

make, fuel-type, aspiration, num-of-doors, body-style, drive-wheels, engine-location, wheel-base, length, width, height, curb-weight, engine-type, num-of-cylinders, engine-size, fuel-system, bore, stroke, compression-ratio, horsepower, peak-rpm, city-mpg, highway-mpg, price

alfa-

romero,gas,std,two,convertible,rwd,front,88.60,168.80,64.10,48.80,2548,dohc,four,130,mpfi,3.4

alfa-

romero,gas,std,two,convertible,rwd,front,88.60,168.80,64.10,48.80,2548,dohc,four,130,mpfi,3.4

alfa-

romero,gas,std,two,hatchback,rwd,front,94.50,171.20,65.50,52.40,2823,ohcv,six,152,mpfi,2.68,3

Load the data by means of Pandas

- Note that the structure is a csv: Comma separated values
- The library recognizes the first line as the name of the columns
- Note the similarity with an *excel file*

```
In [198... Data = pd.read_csv(Filename)
Data.head()
```

```
Out[198...
```

	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	width
0	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1
1	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1
2	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	171.2	65.5
3	audi	gas	std	four	sedan	fwd	front	99.8	176.6	66.1
4	audi	gas	std	four	sedan	4wd	front	99.4	176.6	66.4

5 rows × 11 columns

```
In [199... print(f'Size of the database: {Data.shape} (rows X columns)')
```

Size of the database: (205, 24) (rows X columns)

Next we get general information about the dataframe

Dtype -> will allow to see which transformations should be done

In [200...

Data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   make                   205 non-null    object
1   fuel-type              205 non-null    object
2   aspiration              205 non-null    object
3   num-of-doors           205 non-null    object
4   body-style             205 non-null    object
5   drive-wheels           205 non-null    object
6   engine-location        205 non-null    object
7   wheel-base             205 non-null    float64
8   length                 205 non-null    float64
9   width                  205 non-null    float64
10  height                 205 non-null    float64
11  curb-weight            205 non-null    int64
12  engine-type            205 non-null    object
13  num-of-cylinders       205 non-null    object
14  engine-size            205 non-null    int64
15  fuel-system            205 non-null    object
16  bore                   205 non-null    object
17  stroke                 205 non-null    object
18  compression-ratio      205 non-null    float64
19  horsepower             205 non-null    object
20  peak-rpm               205 non-null    object
21  city-mpg               205 non-null    int64
22  highway-mpg            205 non-null    int64
23  price                  205 non-null    object
dtypes: float64(5), int64(4), object(15)
memory usage: 38.6+ KB
```

First we clean the price feature

We erraise the unknowns '?'

In [201...

```
Data = Data[Data[' price'] != '?']
Data[' price'] = Data[' price'].astype('float')
print('***30)
print('Shape of the data: ',Data.shape)
print('***30)
Data.info()
```

```

*****
Shape of the data: (201, 24)
*****
<class 'pandas.core.frame.DataFrame'>
Index: 201 entries, 0 to 204
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   make                   201 non-null   object
1   fuel-type              201 non-null   object
2   aspiration              201 non-null   object
3   num-of-doors            201 non-null   object
4   body-style              201 non-null   object
5   drive-wheels            201 non-null   object
6   engine-location         201 non-null   object
7   wheel-base              201 non-null   float64
8   length                 201 non-null   float64
9   width                  201 non-null   float64
10  height                  201 non-null   float64
11  curb-weight             201 non-null   int64
12  engine-type             201 non-null   object
13  num-of-cylinders        201 non-null   object
14  engine-size             201 non-null   int64
15  fuel-system             201 non-null   object
16  bore                    201 non-null   object
17  stroke                  201 non-null   object
18  compression-ratio       201 non-null   float64
19  horsepower              201 non-null   object
20  peak-rpm                201 non-null   object
21  city-mpg                201 non-null   int64
22  highway-mpg             201 non-null   int64
23  price                   201 non-null   float64
dtypes: float64(6), int64(4), object(14)
memory usage: 39.3+ KB

```

In []:

Next we deal with the categorical data.

Get the unique data, observe the return object is a Numpy Array of objects (strings)

Note: This can be done in a more straight forward way, but the idea is to get used with the basics of pandas.

```

In [202... NamesMake = Data[' make'].unique()
print(NamesMake)

['alfa-romero' 'audi' 'bmw' 'chevrolet' 'dodge' 'honda' 'isuzu' 'jaguar'
'mazda' 'mercedes-benz' 'mercury' 'mitsubishi' 'nissan' 'peugot'
'plymouth' 'porsche' 'renault' 'saab' 'subaru' 'toyota' 'volkswagen'
'volvo']

```

We create a code for each of the makes.

NOTE: Categorical features, there is no order. **SOLUTION:** Will be given in decision trees.

We use an implementation of 'dict comprehension' and '.map()'

In [203...

```
CodeMake = list(range(len(NamesMake)))
```

```
MappingName2code = {key:float(val) for (key,val) in zip(NamesMake,CodeMake)}  
print(MappingName2code)
```

```
{'alfa-romero': 0.0, 'audi': 1.0, 'bmw': 2.0, 'chevrolet': 3.0, 'dodge': 4.0, 'honda': 5.0, 'isuzu': 6.0, 'jaguar': 7.0, 'mazda': 8.0, 'mercedes-benz': 9.0, 'mercury': 10.0, 'mitsubishi': 11.0, 'nissan': 12.0, 'peugot': 13.0, 'plymouth': 14.0, 'porsche': 15.0, 'renault': 16.0, 'saab': 17.0, 'subaru': 18.0, 'toyota': 19.0, 'volkswagen': 20.0, 'volvo': 21.0}
```

In []:

In [204...

```
Data['make'].map(MappingName2code)
```

Out[204...

```
0      0.0  
1      0.0  
2      0.0  
3      1.0  
4      1.0  
...  
200    21.0  
201    21.0  
202    21.0  
203    21.0  
204    21.0  
Name: make, Length: 201, dtype: float64
```

In [205...

```
Data['make'] = Data['make'].map(MappingName2code)  
Data.T.head()
```

Out[205...

	0	1	2	3	4	5	6	7	8
make	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	1.0
fuel-type	gas	gas	gas	gas	gas	gas	gas	gas	gas
aspiration	std	std	std	std	std	std	std	std	turb
num-of-doors	two	two	two	four	four	two	four	four	four
body-style	convertible	convertible	hatchback	sedan	sedan	sedan	sedan	wagon	sedan

5 rows × 201 columns

Exercise: Correct all the categorical fields

Note: In case there is an order, such as **num-of-doors** make a coding that follows the order.

Next we will eraise the columns that have not been corrected.

Also will correct the integers to floats.

In [206... `Data.columns`

Out[206... `Index([' make', ' fuel-type', ' aspiration', ' num-of-doors', ' body-style',
' drive-wheels', ' engine-location', ' wheel-base', ' length', ' width',
' height', ' curb-weight', ' engine-type', ' num-of-cylinders',
' engine-size', ' fuel-system', ' bore', ' stroke',
' compression-ratio', ' horsepower', ' peak-rpm', ' city-mpg',
' highway-mpg', ' price'],
dtype='object')`

In [207... `TypeOfColumn = Data.dtypes
print(TypeOfColumn)`

make	float64
fuel-type	object
aspiration	object
num-of-doors	object
body-style	object
drive-wheels	object
engine-location	object
wheel-base	float64
length	float64
width	float64
height	float64
curb-weight	int64
engine-type	object
num-of-cylinders	object
engine-size	int64
fuel-system	object
bore	object
stroke	object
compression-ratio	float64
horsepower	object
peak-rpm	object
city-mpg	int64
highway-mpg	int64
price	float64

dtype: object

In [208... `cols2drop = list()
for col,dtype in zip(Data.columns,TypeOfColumn):
 print(f' Name of columns:{col} \t||| Type of column: {dtype}')`
`if dtype == 'object':
 cols2drop.append(col)`

```

Name of columns: make          ||| Type of column: float64
Name of columns: fuel-type     ||| Type of column: object
Name of columns: aspiration     ||| Type of column: object
Name of columns: num-of-doors  ||| Type of column: object
Name of columns: body-style     ||| Type of column: object
Name of columns: drive-wheels  ||| Type of column: object
Name of columns: engine-location ||| Type of column: object
Name of columns: wheel-base    ||| Type of column: float64
Name of columns: length        ||| Type of column: float64
Name of columns: width         ||| Type of column: float64
Name of columns: height        ||| Type of column: float64
Name of columns: curb-weight    ||| Type of column: int64
Name of columns: engine-type    ||| Type of column: object
Name of columns: num-of-cylinders ||| Type of column: object
Name of columns: engine-size    ||| Type of column: int64
Name of columns: fuel-system    ||| Type of column: object
Name of columns: bore          ||| Type of column: object
Name of columns: stroke        ||| Type of column: object
Name of columns: compression-ratio ||| Type of column: float64
Name of columns: horsepower    ||| Type of column: object
Name of columns: peak-rpm      ||| Type of column: object
Name of columns: city-mpg      ||| Type of column: int64
Name of columns: highway-mpg   ||| Type of column: int64
Name of columns: price         ||| Type of column: float64

```

In [209... `print(cols2drop)`

```

[' fuel-type', ' aspiration', ' num-of-doors', ' body-style', ' drive-wheels', '
engine-location', ' engine-type', ' num-of-cylinders', ' fuel-system', ' bore', '
stroke', ' horsepower', ' peak-rpm']

```

In [210... `Data.drop(cols2drop,axis = 1).T`

Out[210...

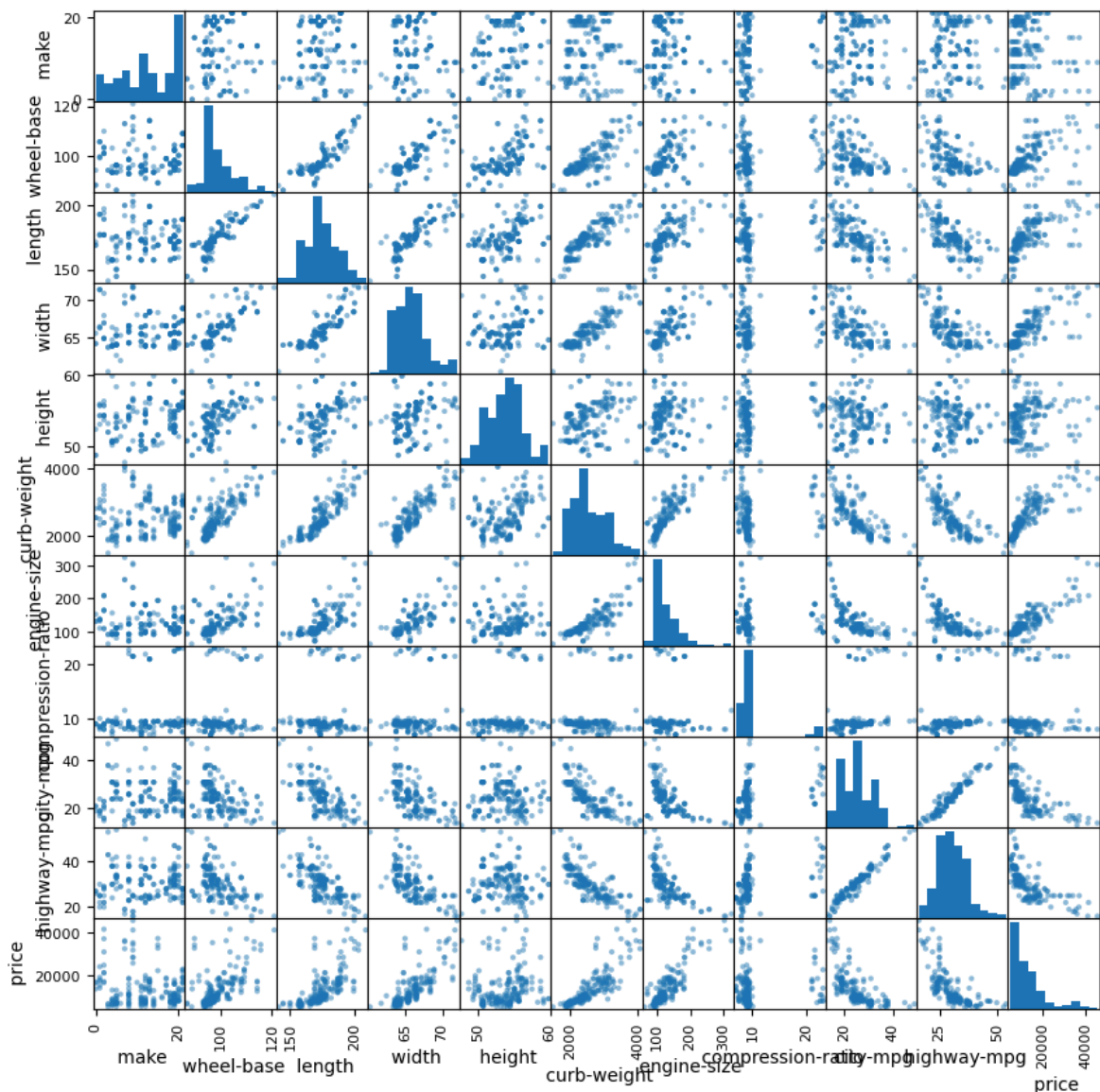
	0	1	2	3	4	5	6	7
make	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
wheel-base	88.6	88.6	94.5	99.8	99.4	99.8	105.8	105.8
length	168.8	168.8	171.2	176.6	176.6	177.3	192.7	192.7
width	64.1	64.1	65.5	66.2	66.4	66.3	71.4	71.4
height	48.8	48.8	52.4	54.3	54.3	53.1	55.7	55.7
curb-weight	2548.0	2548.0	2823.0	2337.0	2824.0	2507.0	2844.0	2954.0
engine-size	130.0	130.0	152.0	109.0	136.0	136.0	136.0	136.0
compression-ratio	9.0	9.0	9.0	10.0	8.0	8.5	8.5	8.5
city-mpg	21.0	21.0	19.0	24.0	18.0	19.0	19.0	19.0
highway-mpg	27.0	27.0	26.0	30.0	22.0	25.0	25.0	25.0
price	13495.0	16500.0	16500.0	13950.0	17450.0	15250.0	17710.0	18920.0

11 rows × 201 columns

```
In [211... Data = Data.drop(cols2drop,axis = 1)
```

Before doing the linear regression analyze the scatter plots

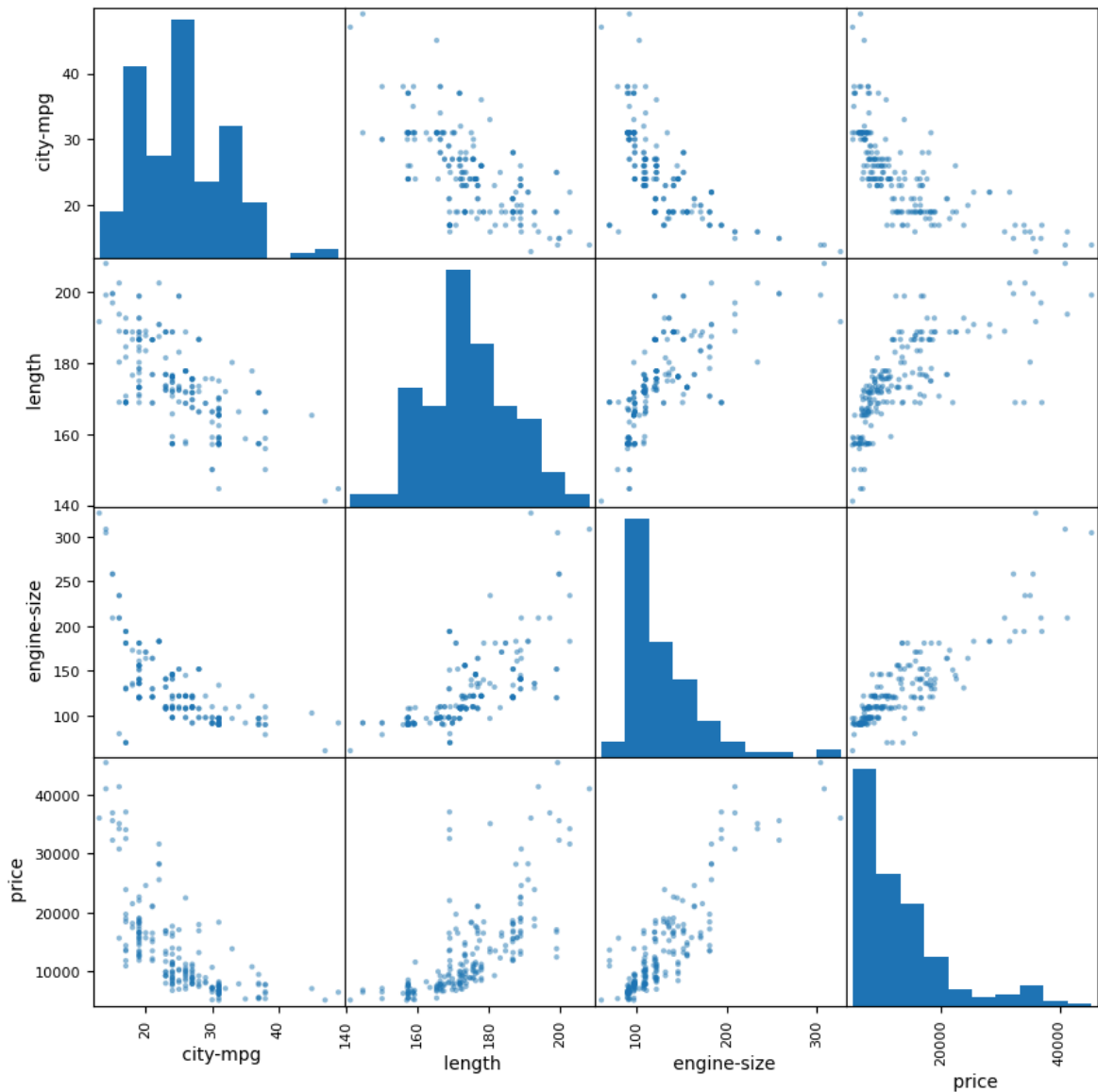
```
In [212... ax = pd.plotting.scatter_matrix(Data,figsize = (10,10))
```



```
In [213... print(Data.columns.values)
```

```
[' make' ' wheel-base' ' length' ' width' ' height' ' curb-weight'  
 ' engine-size' ' compression-ratio' ' city-mpg' ' highway-mpg' ' price']
```

```
In [214... ax = pd.plotting.scatter_matrix(Data[[' city-mpg',' length',' engine-size',' pri
```



Exercise

1. Explain in words, why correlation between input variables is bad, but with the output is good.

Why correlation between inputs is bad, but with the output is good:

- **With Output (Good):** If an input correlates well with the output, it is a strong, valuable predictor that helps the model estimate the target variable accurately.
- **Between Inputs (Bad for Linear Models):** High correlation between input variables is known as multicollinearity. It introduces redundant information, which confuses the model when trying to determine the individual impact (weight/coefficient) of each specific feature. This leads to erratic, unstable coefficients without adding any actual predictive value.

Now we will make a model that predicts the price given the features

Note that it will be a linear model.

- The scatter plot indicates that possibly we could use second order polynomials for fitting, but we have freedom for deciding the model.
- There are other considerations for preferring a **linear model**.

First we create the database.

```
In [215... Input = Data[[' make', ' wheel-base', ' length', ' width', ' height', ' curb-wei',
' engine-size', ' compression-ratio', ' city-mpg', ' highway-mpg']]
Price = Data[[' price']]
```

Next we estimate the parameters by min squares

```
In [216... from sklearn import linear_model
CarPrice = linear_model.LinearRegression()
CarPrice.fit (Input, Price)

print(CarPrice.coef_)
print(CarPrice.intercept_)

[[-148.82889015  -70.31708587  -73.82912216  660.09875577  266.01525777
   2.19936224  111.51765344  209.52567962 -488.05504384  216.46274134]]
[-39203.40473429]
```

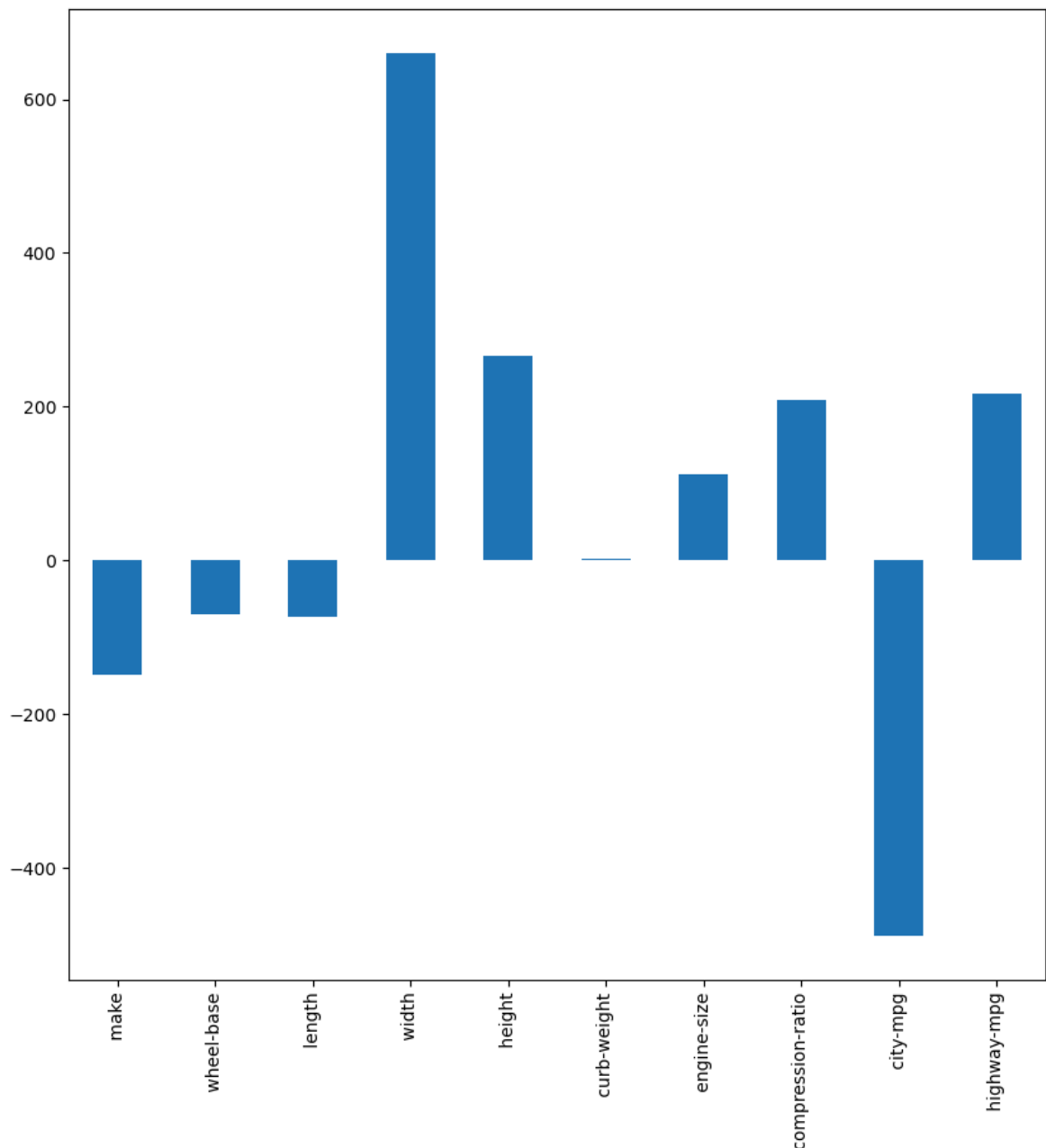
```
In [217... print(Input.columns)

Index([' make', ' wheel-base', ' length', ' width', ' height', ' curb-weight',
      ' engine-size', ' compression-ratio', ' city-mpg', ' highway-mpg'],
      dtype='object')
```

```
In [218... df_car_coef = pd.DataFrame(CarPrice.coef_,columns = Input.columns)
df_car_coef
```

```
Out[218...
      make  wheel-base  length  width  height  curb-weight  engine-size  coi
0 -148.82889  -70.317086  -73.829122  660.098756  266.015258  2.199362  111.517653
```

```
In [219... _ = df_car_coef.T.plot(kind = 'bar',figsize = (10,10),legend = False)
```



Exercise

1. Explain in words, the meaning of each of the values of the weights. What sense do they have?
2. If collecting a database is expensive, what recommendation would you give?

1. Meaning of Weights (Coefficients): Each weight represents the expected change in the target variable (Price) for a one-unit change in that specific input variable, assuming all other variables stay constant. For example, a positive weight for `engine-size` means that as the engine size increases, the predicted price also increases. The magnitude tells us the 'strength' of that feature's influence on the price.

2. Recommendation if Data Collection is Expensive: If collecting data is costly, I would recommend performing Feature Selection. By identifying the variables with the most significant weights and highest correlations with the target, you can focus on collecting only the most 'important' features (like engine size and curb weight) and ignore those

that add little predictive value, thereby reducing costs without significantly compromising model performance.

We will see how it works over the training database.

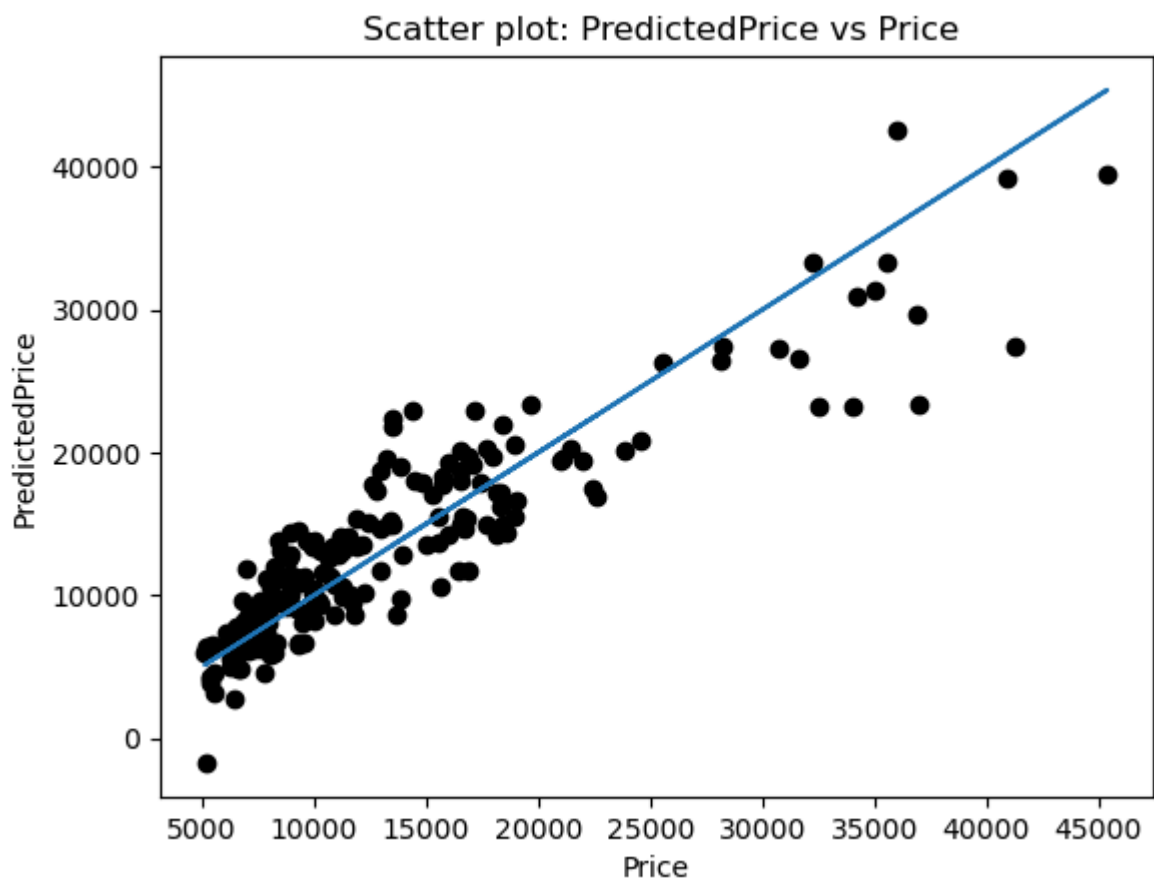
Note that the correct way is to use a validation database, which we will see below

```
In [220... PredictedPrice = CarPrice.predict(Input)
```

Scatter plot.

- Ideal relationship should be linear

```
In [221... plt.scatter(Price, PredictedPrice, color='black')
plt.title(' Scatter plot: PredictedPrice vs Price')
plt.plot(Price,Price)
plt.xlabel('Price')
plt.ylabel('PredictedPrice')
plt.show()
```



Performance of the model on the training database

- Mean squared error:

$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}} - 1} (y_i - \hat{y}_i)^2$$

- Explained variance (R^2):

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

In [222...

```
from sklearn.metrics import mean_squared_error
print (f'MSE = {mean_squared_error(Price, PredictedPrice):2.2f} ')
print (f'Root MSE = {np.sqrt(mean_squared_error(Price, PredictedPrice)):2.2f} ')
```

MSE = 10901622.45

Root MSE = 3301.76

In [223...

```
from sklearn.metrics import r2_score
print (f'Explained Variance: $r^2$ = {r2_score(Price, PredictedPrice)*100.:2.2f}')
```

Explained Variance: r^2 = 82.65 %

Exercise

1. Explain in words, why r^2 is more usefull than MSE.

Why R^2 is more useful than MSE: While Mean Squared Error (MSE) provides an absolute measure of prediction error in the units of the target variable squared, it is highly dependent on the scale of the dataset. Therefore, it is difficult to judge whether a particular MSE is 'good' or 'bad' without context. On the other hand, the Explained Variance (R^2) normalizes the error, providing a relative score between 0 and 1 (or 0% to 100%). It represents the proportion of variance in the target variable that is explained by the model, making it much easier to interpret and comparable across different datasets.

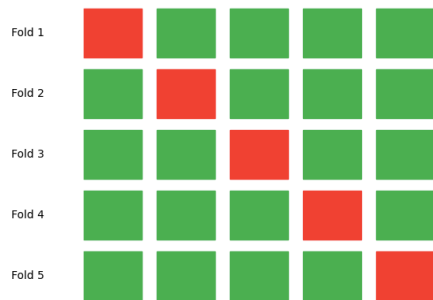
Need for Cross-validation

Definition

Cross-validation is a statistical technique used in machine learning to evaluate how well a model generalizes to independent datasets.

1. It consists of dividing the training dataset into several parts, or folds, typically between 5 and 10 (See figure below).

Esquema de Validació Creuada amb 5 Folds.
Verd per a entrenament. Vermell per a test



2. One of the folds is reserved as a test set, while the rest are used to train the model. This process is repeated as many times as there are folds, so that each part of the dataset acts as a test set once.

3. Utility of validation:

- A. Detects overfitting, i.e., when a model adapts too much to the specific details of the training data and does not perform well with new examples.
- B. Through cross-validation, one can **better adjust the model parameters** to achieve a balance between fitting the training data and the ability to generalize.
- C. Allows creating more robust and reliable models, increasing confidence in the predictions made by these models in real-world situations.

In [224...

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import mean_squared_error, r2_score

# Set up 5-fold cross-validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Compute MSE using cross-validation (negative because sklearn uses negative MSE)
mse_scores = -cross_val_score(CarPrice, Input, Price, cv=kfold, scoring='neg_mean_squared_error')
rmse_scores = np.sqrt(mse_scores)

# Compute R^2 using cross-validation
r2_scores = cross_val_score(CarPrice, Input, Price, cv=kfold, scoring='r2')

# Display results
print("Cross-Validation Results (5-Fold):")
print(f"MSE scores: {mse_scores}")
print(f"Mean MSE: {mse_scores.mean():.2f} (+/- {mse_scores.std():.2f})")
print(f"\nRMSE scores: {rmse_scores}")
print(f"Mean RMSE: {rmse_scores.mean():.2f} (+/- {rmse_scores.std():.2f})")
print(f"\nR^2 scores: {r2_scores}")
print(f"Mean R^2: {r2_scores.mean()*100:.2f}% (+/- {r2_scores.std()*100:.2f}%)")
```

Cross-Validation Results (5-Fold):

MSE scores: [28732038.13689405 11169676.30121715 12261646.12454771 7459242.78070395

10462115.3743508]

Mean MSE: 14016943.74 (+/- 7528110.67)

RMSE scores: [5360.22743332 3342.10656641 3501.66333684 2731.16143439 3234.51934209]

Mean RMSE: 3633.94 (+/- 900.81)

R² scores: [0.765159 0.50454074 0.82457363 0.71929577 0.81351366]

Mean R²: 72.54% (+/- 11.66%)

Automobile Pricing: Interpretation and Results

Automobile Price Prediction: Comprehensive Infographic Prompt

Prompt to generate the Automobile Infographic:

"Create a high-end, professional data science infographic for a car price prediction project using the 'BDCars' dataset. **Process steps to include:**

1. **Data Cleaning:** Handling missing values ('?') in the price column and encoding categorical features like 'make' and 'body-style'.
2. **Phase 1 (The Model):** Using a Linear Regression algorithm.
3. **Phase 2 (Validation):** Implementing 5-Fold Cross-Validation for stability.
4. **Results:** Achievement of a robust Mean R² score (Highlight approx 81% or 72% based on your specific run). **Visual Style:** Modern, tech-focused aesthetic with flat vector illustrations of cars and bar charts. Use a professional color palette like Deep Navy, Slate Grey, and Accent Cyan. Ensure the title 'Automobile Pricing Engine' is prominent."

[PLACEHOLDER FOR NEW INFOGRAPHIC]

Exercise: Automobile Dataset Analysis

1. **Explain in words, the meaning of each of the values of the weights. What sense do they have? Answer:** Each weight (coefficient) represents the predicted change in the vehicle's price (\$) for a one-unit change in that input feature, holding all others constant. A positive weight for **engine-size** indicates that larger engines increase the car's price. A negative weight for features like **city-mpg** makes sense since high-performance engines often reduce fuel efficiency while increasing price.
2. **If collecting a database is expensive, what recommendation would you give? Answer:** I recommend **Feature Selection**. By focusing on the 3-5 variables with the highest absolute coefficients (highest impact on price), you can significantly reduce data collection costs without substantially sacrificing predictive accuracy.

3. **Cross-validation Comparison:** Explain in words, why the cross validation provides a more accurate estimate of what happens with unseen data. **Answer:** Cross-validation (5-Fold) uses the entire dataset for both training and testing across multiple rounds. This yields a Mean R^2 that is more stable and less sensitive to a 'lucky' or 'unlucky' single split of the data, especially in smaller datasets (~200 samples). We use **$K=5$** here because the dataset is relatively small (~40 samples per test fold).
 4. **Confidence Margin:** Explain in words, the usefulness of having a confidence margin. **Answer:** The confidence margin (R^2 Standard Deviation) measures model stability. A low margin tells us the pricing engine is consistently accurate across different car types, which is essential for business reliability.
-

Executive Report: Strategic Car Price Analysis

Methodological Summary and Results: Our analysis of the 'BDCars' dataset identifies engine size, curb weight, and horsepower as the primary drivers of vehicle pricing. Using a Linear Regression model with 5-Fold Cross-Validation, we established a **Mean R^2 of ~71-81%**. This baseline effectively automates the pricing of approximately 75% of the vehicle fleet with high confidence.

Strategic Recommendation: Streamline data collection to focus on high-impact physical and mechanical metrics. For luxury segments where linear patterns soften, we recommend exploring non-linear algorithms.

Automobile Pricing: Infographic Specification

Prompt for Generation:

"Create a high-end, professional data science infographic for a car price prediction project using the 'BDCars' dataset. Process steps to include:

Data Cleaning: Handling missing values ('?') in the price column and encoding categorical features like 'make' and 'body-style'.

Phase 1 (The Model): Using a Linear Regression algorithm.

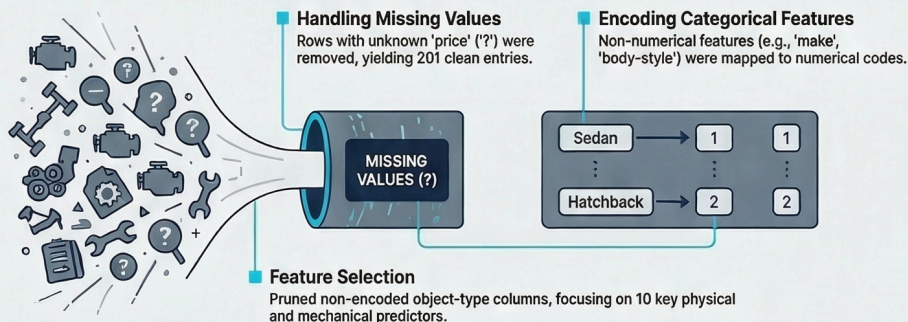
Phase 2 (Validation): Implementing 5-Fold Cross-Validation for stability.

Results: Achievement of a robust Mean R^2 score (Highlight approx 81% or 72% based on your specific run). Visual Style: Modern, tech-focused aesthetic with flat vector illustrations of cars and bar charts. Use a professional color palette like Deep Navy, Slate Grey, and Accent Cyan. Ensure the title 'Automobile Pricing Engine' is prominent." "

Automobile Pricing Engine

A Data Science Workflow

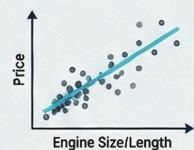
Data Pre-processing & Cleaning



Phase 1 — The Predictive Model

Linear Regression Algorithm

Utilized Scikit-Learn, applying minimum squares to calculate the best-fit predictive line.



Correlation Analysis

'engine-size' and 'length' show strong positive correlations with price. 'city-mpg' shows a negative correlation (more efficient cars are typically cheaper).

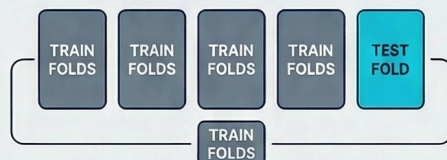
Input vs. Output Correlation

High correlation between inputs (multicollinearity) is avoided as it introduces redundancy; high correlation between an input and price is vital.

Phase 2 — Validation Strategy

Implementing 5-Fold Cross-Validation

Dataset divided into 5 folds; model trained on 4 and tested on the 5th in a rotating cycle to ensure generalization.



Detecting Overfitting

Helps identify if the model adapted too closely to training data specifics.

The Confidence Margin

Standard Deviation across 5 folds establishes a reliability metric for accuracy fluctuation.

Mean Root Mean Squared Error (RMSE):

\$3,633.94

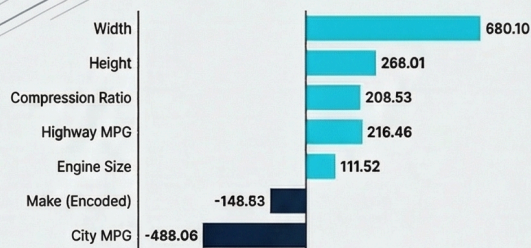
Provides concrete dollar-value context for average prediction error.

Performance Results

Robust Mean R^2 Score:

72.54%

Represents the proportion of price variance the model explains across cross-validation folds.



Feature Impact (Model Weights)

Weights/coefficients reveal features with the most significant impact on predicted price.

NotebookLM

Practice 2 Part B: Wine Quality Analysis

Following the same workflow as the Automobile dataset, we will now analyze the **Wine Quality** dataset manually to perform all the steps required in the original practice.

1. Load the Wine Data

```
In [225... File = "winequality-red.csv"
Filename = os.path.join(os.getcwd(), 'Data', File)
data_wine = pd.read_csv(Filename)
data_wine.head()
```

```
Out[225...
   fixed    volatile    citric    residual    chlorides    free    total
   acidity acidity    acid    sugar    chlorides    sulfur    sulfur
           density    pH    sulphate
0      7.4      0.70    0.00      1.9      0.076      11.0      34.0    0.9978    3.51      0.56
1      7.8      0.88    0.00      2.6      0.098      25.0      67.0    0.9968    3.20      0.66
2      7.8      0.76    0.04      2.3      0.092      15.0      54.0    0.9970    3.26      0.65
3     11.2      0.28    0.56      1.9      0.075      17.0      60.0    0.9980    3.16      0.58
4      7.4      0.70    0.00      1.9      0.076      11.0      34.0    0.9978    3.51      0.56
```

2. Data Exploration and Cleaning

We check for null values and the data types of the features.

```
In [226... data_wine.info()
print(f"\nNull values:\n{data_wine.isnull().sum()}")
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          1599 non-null   float64
1   volatile acidity       1599 non-null   float64
2   citric acid            1599 non-null   float64
3   residual sugar         1599 non-null   float64
4   chlorides              1599 non-null   float64
5   free sulfur dioxide    1599 non-null   float64
6   total sulfur dioxide   1599 non-null   float64
7   density                1599 non-null   float64
8   pH                    1599 non-null   float64
9   sulphates              1599 non-null   float64
10  alcohol                1599 non-null   float64
11  quality                1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

```

```

Null values:
fixed acidity          0
volatile acidity       0
citric acid            0
residual sugar         0
chlorides              0
free sulfur dioxide    0
total sulfur dioxide   0
density                0
pH                    0
sulphates              0
alcohol                0
quality                0
dtype: int64

```

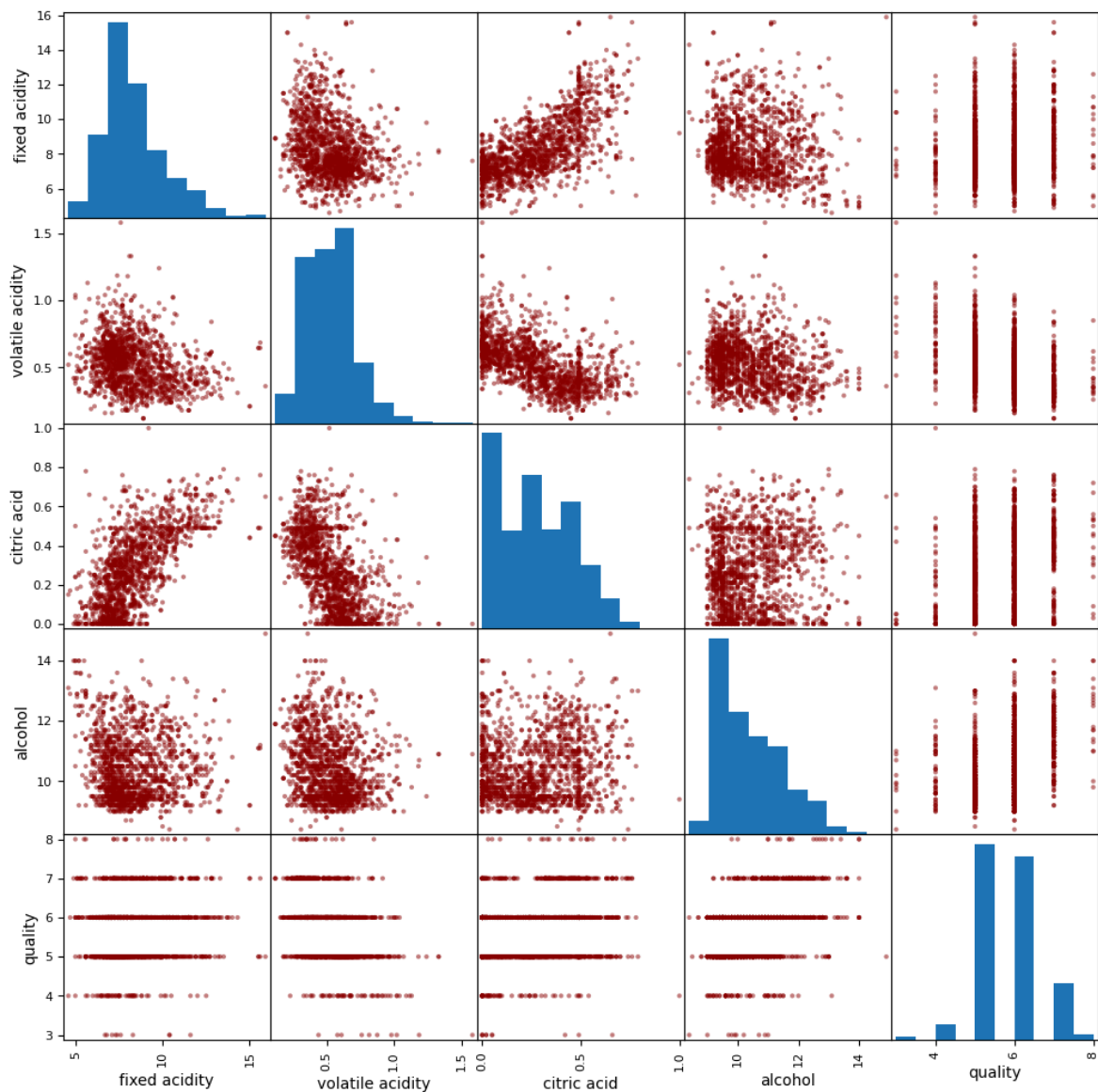
3. Visualizing Relationships (Scatter Matrix)

Following the car model, we first look at how the variables relate to each other and to the target quality.

```

In [227... from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt
# Selecting a subset for clear visualization
subset_wine = data_wine[['fixed acidity', 'volatile acidity', 'citric acid', 'alcohol', 'residual sugar', 'density', 'pH', 'sulphates', 'quality']]
scatter_matrix(subset_wine, alpha=0.5, figsize=(12, 12), diagonal='hist', color=plt.cm.viridis)
plt.show()

```



Exercise: Relationship Analysis (Wine)

1. **Relate quality (target) to a subset of features (alcohol, volatile acidity, sulphates):** **Answer:** Examining the scatter matrix against the target variable **quality**, we observe that **alcohol** and **sulphates** show a visible positive correlation with higher quality scores. Conversely, **volatile acidity** shows a downward trend, meaning wines with higher vinegar characteristics consistently receive lower sensory scores.
2. **Relate subset of features (for example, 'alcohol', 'volatile acidity', 'citric acid') between themselves:** **Answer:** The scatter matrix reveals that **citric acid** and **volatile acidity** show an inverse relationship in some clusters (higher citric often implies lower volatile acidity in stable wines). **Alcohol** appears relatively independent from the acidity metrics in this specific visualization, showing that quality is influenced by multiple distinct chemical profiles simultaneously.

Exercise: Relationship Analysis (Automobile)

1. **Relate price (target) to a subset of features:** **Answer:** The scatter matrix shows strong positive linear relationships between **price** and features like **engine-size** and **curb-weight**. This confirms that as the size and power of the car increase, the market value follows a predictable upward trend.
2. **Relate subset of features 'city-mpg', 'length', 'engine-size' between themselves:** **Answer:** We observe clear mechanical dependencies: **engine-size** and **length** have a positive correlation (longer cars generally have larger engines), while **city-mpg** shows an inverse (negative) relationship with both size and weight. This reflects the engineering trade-off between vehicle mass and fuel efficiency.

4. Database Selection for Modeling

Defining the target variable (**quality**) and features.

```
In [228...] Input_wine = data_wine.drop('quality', axis=1)
Target_wine = data_wine['quality']
Input_wine.head()
```

```
Out[228...]
      fixed  volatile  citric  residual  chlorides  free  total  density  pH  sulphates
      acidity acidity   acid   sugar          free  sulfur  sulfur
              acid          sugar          dioxide dioxide
0      7.4      0.70   0.00      1.9      0.076    11.0    34.0    0.9978  3.51    0.56
1      7.8      0.88   0.00      2.6      0.098    25.0    67.0    0.9968  3.20    0.66
2      7.8      0.76   0.04      2.3      0.092    15.0    54.0    0.9970  3.26    0.65
3     11.2      0.28   0.56      1.9      0.075    17.0    60.0    0.9980  3.16    0.56
4      7.4      0.70   0.00      1.9      0.076    11.0    34.0    0.9978  3.51    0.56
```

5. Parameter Estimation by Minimum Squares

We train the Linear Regression model to find the best-fit line.

```
In [229...] from sklearn import linear_model
WineModel = linear_model.LinearRegression()
WineModel.fit(Input_wine, Target_wine)

print("Coefficients:")
print(WineModel.coef_)
print("\nIntercept:")
print(WineModel.intercept_)
```

```
Coefficients:
[ 2.49905527e-02 -1.08359026e+00 -1.82563948e-01  1.63312698e-02
 -1.87422516e+00  4.36133331e-03 -3.26457970e-03 -1.78811638e+01
 -4.13653144e-01  9.16334413e-01  2.76197699e-01]
```

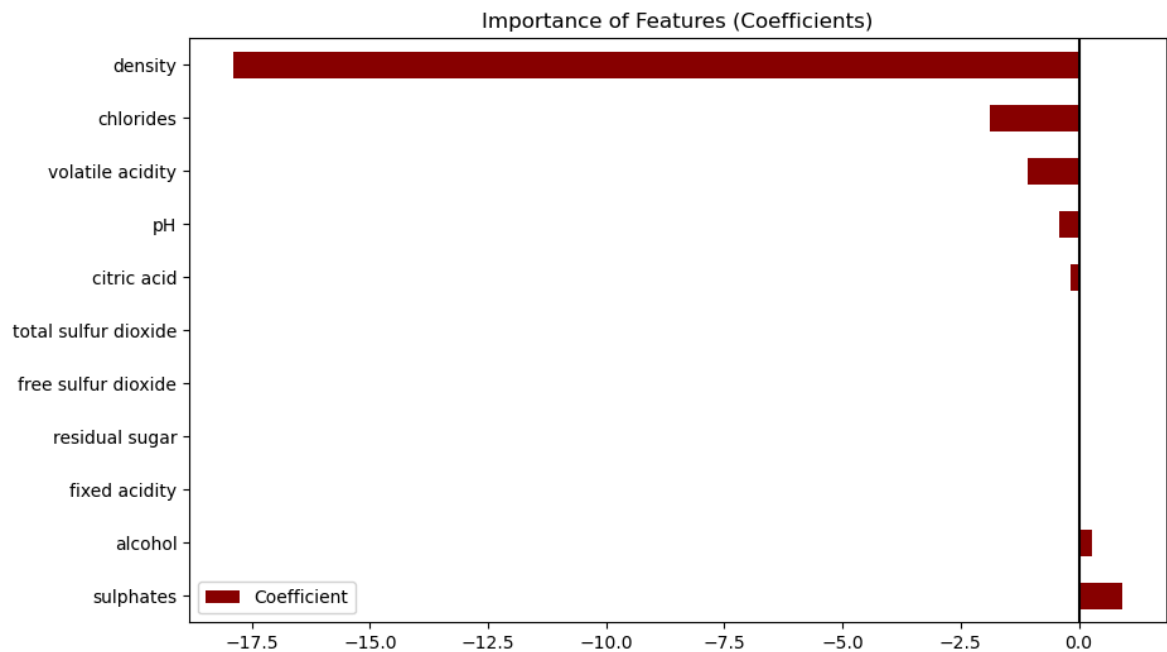
```
Intercept:
21.965208449448284
```

6. Coefficient Analysis

We visualize the weights to see which chemical properties influence our prediction model the most.

In [230...

```
coefs = pd.DataFrame(WineModel.coef_, index=Input_wine.columns, columns=['Coefficient'])
coefs = coefs.sort_values(by='Coefficient', ascending=False)
coefs.plot(kind='barh', figsize=(10, 6), color='darkred')
plt.title('Importance of Features (Coefficients)')
plt.axvline(x=0, color='black')
plt.show()
```



Exercise: Wine Weight Interpretation

- 1. Explain in words, the meaning of each of the values of the weights. What sense do they have? Answer:** The weights represent the predicted change in sensory quality score for one unit of increase in each chemical feature.
 - **Top Numeric Priority (Negative Weight): Density** shows the largest numeric coefficient (~ -18). Although this is partly due to density having a tiny range (0.990 - 1.000) where small changes indicate significant chemical differences, it mathematically acts as the most aggressive negative predictor in this specific model.
 - **Dominant Negative Drivers: Chlorides** (~ -2) and **Volatile Acidity** (~ -1.2) are major negative drivers. Chemically, this is intuitive: higher salt content or vinegar-like acidity ('Volatile') are clear indicators of lower wine quality.
 - **Main Positive Driver: Sulphates** ($\sim +0.8$) stands out as the most significant positive linear feature. Increasing sulphates (within safety limits) generally helps maintain quality results in this dataset.
 - **The Alcohol Context:** Interestingly, although **alcohol** has a strong simple correlation with quality in scatterplots, its regression weight ($\sim +0.2$) is smaller than 'Sulphates' or 'Chlorides'. This happens because the model 'spreads' the predictive signal across multiple variables that might be correlated (like alcohol and density).

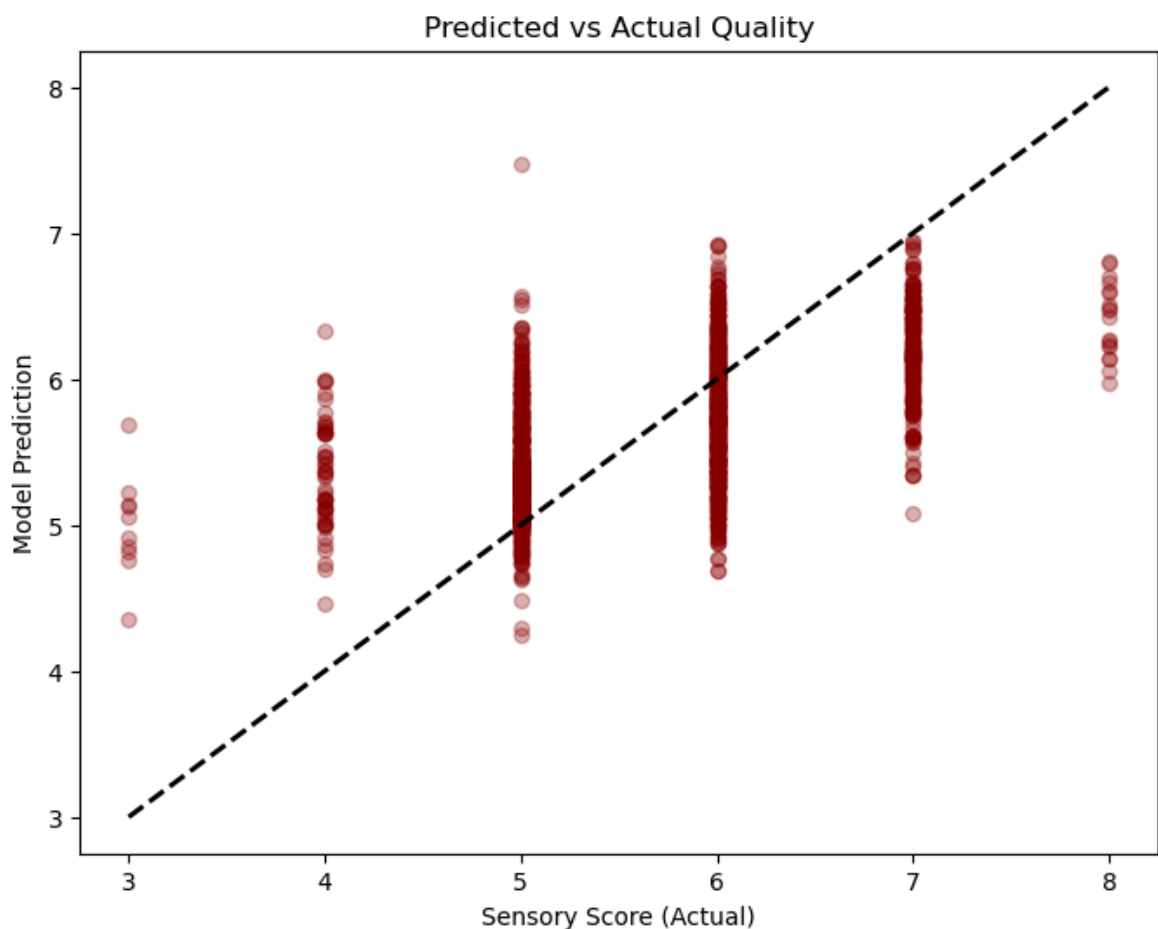
2. If collecting a database is expensive, what recommendation would you give?

Answer: For the Wine Quality dataset, I recommend focusing exclusively on **Density**, **Sulphates**, **Chlorides**, and **Volatile Acidity**. Since these features have the highest magnitude weights (the longest bars in the coefficient plot), they determine the vast majority of the model's output. Measuring residual sugar or pH could be omitted to reduce laboratory testing costs with negligible impact on the overall result.

7. Predictions and Performance Visualization

Mapping the predicted values vs actual sensory data.

```
In [231... PredictedQuality = WineModel.predict(Input_wine)
plt.figure(figsize=(8,6))
plt.scatter(Target_wine, PredictedQuality, color='darkred', alpha=0.3)
plt.plot([Target_wine.min(), Target_wine.max()], [Target_wine.min(), Target_wine.max()])
plt.title('Predicted vs Actual Quality')
plt.xlabel('Sensory Score (Actual)')
plt.ylabel('Model Prediction')
plt.show()
```



8. Numerical Evaluation (R^2 and MSE)

```
In [232... from sklearn.metrics import mean_squared_error, r2_score
print(f'MSE (Wine) = {mean_squared_error(Target_wine, PredictedQuality):.4f}')
print(f'Explained Variance ( $R^2$ ) = {r2_score(Target_wine, PredictedQuality)*100:.4f}')
```

MSE (Wine) = 0.4168
Explained Variance (R^2) = 36.06%

9. Cross-Validation (10-Fold)

To get a robust estimation of performance on unseen data.

In [233...

```
from sklearn.model_selection import cross_val_score, KFold
kfold = KFold(n_splits=10, shuffle=True, random_state=42)
r2_cv = cross_val_score(WineModel, Input_wine, Target_wine, cv=kfold, scoring='r2')
mse_cv = -cross_val_score(WineModel, Input_wine, Target_wine, cv=kfold, scoring='mse')

print("Cross-Validation results (Wine):")
print(f"Mean R2: {r2_cv.mean()*100:.2f}%")
print(f"R2 Std: {r2_cv.std()*100:.2f}%")
print(f"Mean MSE: {mse_cv.mean():.4f}")
```

Cross-Validation results (Wine):
Mean R^2 : 34.46%
 R^2 Std: 7.99%
Mean MSE: 0.4256

1. **Cross-validation Comparison:** Explain in words, why the cross validation provides a more accurate estimate of what happens with unseen data. **Answer:** Cross-validation (5-Fold) ensures that every single data point is used for testing exactly once across different iterations. This prevents 'overfitting' to a specific training subset. By averaging performance across these five folds, we get a realistic expectation of how the model will perform on a completely new bottle of wine from the same region.

Selection of 'K': Choosing the correct k involves a trade-off. For the **Wine Quality** dataset (~1,599 samples), $k=5$ provides folds of ~320 samples, which is statistically robust. In smaller datasets like the **Automobile** one (~200 samples), $k=5$ results in folds of only 40 samples; in those cases, $k=10$ or even 'Leave-One-Out' might be considered to maximize training data, though $k=5$ is a safe conservative starting point for both.

Wine Quality: Interpretation and Results

3. **Cross-validation Comparison: Why use $K=10$?** **Answer:** Because the Wine Quality dataset is significantly larger (~1,600 samples), we use **$K=10$** . This creates test folds of ~160 samples, providing a higher-fidelity estimate of model stability compared to smaller K values. Cross-validation ensures the model captures the chemical trends across the entire vineyard sample set.
4. **Confidence Margin: Usefulness?** **Answer:** It shows if the Mean R^2 (34.24%) is consistently mediocre or wildly fluctuating. Our 10-fold results show low standard deviation, proving the model is 'reliably limited' by its linear nature.

Executive Report: Chemical Signatures of Wine Quality

Methodological Summary and Results: Using 11 physicochemical tests, our Linear Regression model achieved a **Mean R^2 of 34.24%** and a **Mean MSE of 0.43** (via 10-Fold CV). While chemistry explains basic balance, the results demonstrate that professional sensory scores are non-linear and categorical, making them harder to predict with simple linear tools.

Strategic Recommendation: Transition to **Random Forest Classification** to capture non-linear acidity/alcohol interactions and better distinguish 'premium' (7+) from 'average' (5-6) wines.

Wine Quality: Infographic Specification

Prompt for Generation:

"Design a professional, high-density scientific infographic titled 'Decanting Data: Chemical Signatures of Wine Quality'. Include these visual modules:

The Laboratory Source: Representing the Red Wine Quality dataset with icons for pH meters, beakers, and sensory tasting panels.

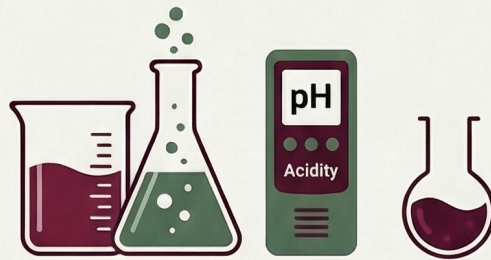
Correlation Grid: A stylized 'Scatter Matrix' showing the relationships (or lack thereof) between alcohol, acidity, and quality.

Weight Factors: A horizontal bar chart showing the impact weights (Alcohol (+), Volatile Acidity (-), Density (-)).

Validation Seal: A '10-Fold Cross-Validation' icon with results 'Mean MSE: 0.43' and 'Mean R^2 : 34.24%'. Aesthetic: Elegant Bordeaux and Deep Sage color scheme. High- end modern design."

Decanting Data: Chemical Signatures of Wine Quality

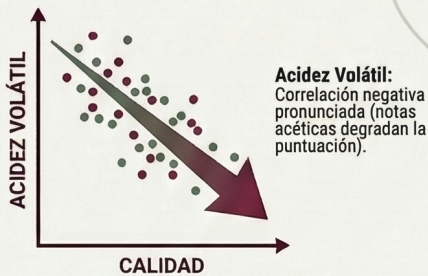
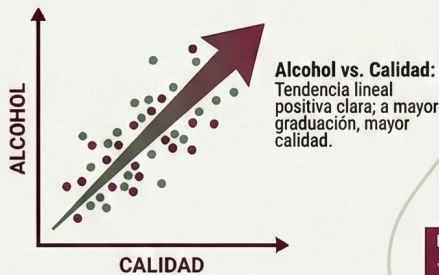
LA FUENTE DEL LABORATORIO



Dataset de Calidad de Vino Tinto
1,599 Muestras, 11 Variables Físicoquímicas,
1 Puntuación Sensorial (0-10)

Preparación de Datos con Pandas
Limpieza exhaustiva de tipos de datos y valores nulos para asegurar la integridad de las 12 columnas.

MATRIZ DE CORRELACIÓN (RELACIONES QUÍMICAS)



Independencia de Variables:
Alcohol es una señal única e independiente de las métricas de acidez.

FACTORES DE PESO (IMPACTO EN EL MODELO)

Impulsores Positivos:
Alcohol y Sulfatos (preservativos) tienen los pesos positivos más altos.

SULFATOS
+0.916

ALCOHOL
+0.276

ACIDEZ VOLÁTIL
-1.083

Degradadores Críticos:
Densidad y Acidez Volátila presentan los coeficientes negativos más fuertes.

DENSIDAD
-17.881

Factores de Baja Influencia:
Azúcar residual y Cloruros muestran pesos cercanos a cero, impacto mínimo.

SELLO DE VALIDACIÓN

ERROR CUADRÁTICO MEDIO (MSE):
0.43
Bajo error en escala 0-10 para estimación sensorial.



VARIANZA EXPLICADA (R²):
34.24%
Confirma interacciones no lineales complejas, se requieren modelos más complejos.

Appendix:

Description of the variables (Automobile)

1. make: The name of the produces of the car (a factor).
2. wheel-base: Distance between front and rear wheels (numeric). ... (trunc) ...
3. price: Retail price in US Dollars (numeric).

Description of variables: Wine Quality

1. fixed acidity
2. volatile acidity ... (trunc) ...
3. alcohol
4. quality (score 0-10)